

Sisteme informatice:

- = oameni + masini care produc si/sau folosesc informatii care sunt unite prin **sisteme de comunicatii**
- E integrat daca:
 - **Procesele de afaceri** si **procesele informatice** sunt corelate in profunzime
 - Legatura intre diferite programe e in mare masura **automatizata**
 - **Datele** sunt achizitionate si distribuite la toate programele
- Redau procesele productive, interne si schimburile din interiorul firmei

Siste informatice INTEGRATE:

- Pe vertical: Apple -> el face si hardware ul si software ul
- Pe orizontala: Google -> el are Android-ul

Procesul dezvoltarii de software:

- Descrie activitatile ce trebuie derulate si rezultatele (**artefacte**)
- Pentru fiecare activitate se definesc roluri pentru angajati care folosesc metode, directive, conventii, modele
- CERINTA:
 - Conditie necesara unui utilizator pentru a indeplini un obiectiv
 - Conditie care trebuie indeplinita/ componenta care satisface un contract standard
 - Reprezentarea documentata a unei conditii
 - TIPURI:
 - FUNCTIONALE: definesc functionalitatea ceruta de utilizator
 - NON-FUNCTIONALE: de calitate:
 - Utilizare: stabilitate, performanta, scalabilitate, etc.
 - Dezvoltare: mentenanta, extensibilitate, etc.
 - SPECIFICE DOMENIULUI: medical, financiar
- **WATERFALL:**
 - Etape succesive: specificare cerintelor, proiectarea sistemului, implementarea software, integrare si testare, mentenanta
 - DEZAVANTAJE:
 - Necesita specificatii complete
 - Integrare si testare tarzie
 - Planuri si termene nerealiste
 - Nu se foloseste in forma pura
- **AGILE:**
 - Etape succesive: Colectrea cerintelor, proiectarea sistemului, dezvoltare, testare, distributie, feedback
 - AVANTAJE:
 - Permite modificarea cerintelor
 - Integrare si testare continua
 - Planuri, termene realiste

- **ETAPE SI PRINCIPII DE BAZA: intelegerea problemei si cerintelor** functionale (business) si non functionale(performanta), alegerea unei **arhitecturii**, aplicarea **principiilor arhitecturale** (design patterns), alegerea unui **framework**, **documentarea** solutiei, **actualizarea** periodica a solutiei

Arhitectura:

- Esentiala pentru definirea functionalitatilor de implementare:
 - Definirea componentelor sistemului si a interactiunii dintre ele
 - Alegerea unor solutii standard
 - Dezvoltarea unor solutii mai robuste si mentenabile
- **COMPONENTE ARHITECTURALE:**
 - View (V) - afisare continut si interfata utilizator
 - Model (M) - defineste nivelul de date
 - Controller ©, Presenter (P), View-Model(VM) - niveluri de legatura dintre V si M
 - Mvp (Android)
- Modele: de business, de sistem, de aplicatie

Dezvoltarea software:

- Design Patterns
 - = solutii sablon pentru probleme comune
 - Creational: factory, builder, **singleton**
 - Structurale: adapter, bridge, **facade** (furnizeaza o interfata simplificata pentru o structura complexa de clase)
 - Comportamentale: command, iterator, visitor, **observer** (transmite evenimente care alte obiecte)
- Framework
 - permit re folosirea de resurse și oferă un set standard de arhitecturi de aplicații ce sunt gata proiectate și testate
 - Tipuri:
 - De aplicatii: react, angular, etc
 - De planificare: diagrama gantt, wbs
- Modele:
 - **Conceptual:** termeni de business
 - **Tehnic:** termeni tehnici

Integrarea:

- Proces de inginerie care creeaza sau imbunatateste fluxul de informatii intre sisteme informationale (proces de interconectare a unui sistem)
- **STRATEGII:**
 - blue/green deployment: rularea simultana pe 2 instnate si activarea uneia dintre ele
 - Feature toggles: activarea/dezactivarea functionalitatii in productie
 - a/b testing: evaluarea comparative
 - Canary releases: testare versiune noua cu un grup select

- CI/ CD
 - = Integrarea continua/ Distributie continua
 - Ciclu de viata al produsului: planificare, programare, integrare, testare, lansare, distributie, operare, monitorizare
- Organizatiile trebuie sa inteleaga **procesele de afaceri** cat si **datele**
- **NIVELURI DE INTEGRARE:**
 - **De date:**
 - Procesul de transfer al datelor intre diferite locatii (extragere info, prelucrare info, actualizare info)
 - Sursa datelor: baza de date relationala, orientata pe obiecte, multidimensionala
 - **Beneficii**
 - **cost scazut** fata de alte metode de integrare
 - Nu presupune modificari majore de cod
 - **Dificultati**
 - Cand se folosesc concomitent diferite tipuri de baze de date
 - CEL MAI FRECVENT
 - BAZA DE DATE:
 - Relationale (SQL)
 - Nerelationale: documente, grafuri, cheie-valoare
 - Serii de timp
 - **Functional** (logica aplicatie)
 - Utilizarea interfetelor expuse de aplicatii pentru a **accesa datele** si **procesele de afaceri**
 - Frecvent pentru integrarea aplicatiilor **incapsulate** care expun interfete catre procese si date
 - BROKER DE MESAJE
 - **AL INTERFETELOR:**
 - Sunt interfetele expuse de dezvoltatori pentru a accesa dif niveluri sau servicii
 - **AL METODELOR**
 - Fol in comun a unei metode de catre mai multe aplicatii
 - Se pot folosi in comun procese de afaceri
 - Framework-uri, obiecte distribuite, servere partajate
 - **API =** mecanism bine definit pentru conectarea la una sau mai multe resurse
 - **Principala metoda de integrare**
 - HTTP (REST)
 - Client-server
 - Pe mai multe niveluri
 - Nu pastreaz stare
 - SOAP
 - Mesaje XML prin HTTP
 - Transmitere ultra securizata

- GraphQL
 - Cereri structurate prin sitaxa
 - Cereri complexe
- RPC
 - Transmitere de comenzi catre un sistem remote
- **Interfata utilizatorului**
 - Cumularea mai multor interfete utilizator
 - Pentru aplicatii care nu au acces la nivelul bazei de date sau proceselor de afaceri
 - Nu necesita modificari ale aplicatiilor integrate
- **PE ORIZONTALA**
 - Folosirea tehnologiilor sistemelor distribuite care permit schimbul de date si info intre sisteme si subsisteme
- **PE VERTICALA**
 - Folosirea tehnologiilor avansate pentru cresterea performantei
 - Integrarea in cadrul sub istemului
- **BIDIRECTIONALA**
 - **Forward engineering** - rezultatele activitatii la un nivel de abstractizare mai mare sunt trnsmise pe niveluri de executie
 - **Reverse engineering** - regasirea info de modelare dupa efectuarea unor modificari

	API	SDK	Biblioteca	Framework
Scop	Describe interacțiunile dintre componente /aplicații diferite	Abstractizează implementările specifice	Încapsulează funcții și secvențe de cod reutilizabile	Modelează o structură complexă de biblioteci /servicii software
Utilizare	Aplicațiile interacționează prin API-uri	Aplicațiile includ SDK-uri	Aplicațiile includ biblioteci	Aplicațiile sunt construite pe baza unui Framework
Structură	Colecție de endpoint-uri de tip request – response	Colecție de instrumente și biblioteci specifice platformei	Colecție de funcții, clase, interfețe	Colecție de biblioteci și modele arhitecturale structurate
Exemple	• Google Maps API • ChatGPT API • Twitter API	• Android SDK • Java Development Kit	• Chart.js • Socket.io • Bootstrap	• .NET Framework • Spring Boot • Angular

Sisteme software:

- Sistem:
 - = ansamblu de software, hardware si de resure umane care realizeaza functii alea **procesului de afaceri**
 - Poate fi: **pre-proiectat** sau **ad-hoc**
- Proiectarea sistemului:
 - = specificatie a structurii sistemului

- O descompunere a functiilor sistemului in secvente de subfunctii
- **Componenta** = un (sub)sistem
- **Proiectarea unei componente reutilizabile**
 - Definirea **fluxului** de informatii si a **interfetelor** explicite folosite
 - Descrierea posibilitatilor de configurare pentru a expune mai multe variante comportamentale si interfete
- **Comportament**
 - Ceea ce **executa** sistemul/ componenta/ **activitatile** unei componente
 - Poate fi influentat de **conditii de mediu**, **actiuni din trecut** sau de **configurare**
- **Functionalitate**
 - Partea **comportamentului** unei componente care satisface anumite **obiective/scopuri**
 - Comportament = ser de tranzitii ale starii in mediu observat => functia = subset al acestor tranzatii
 - **Rezultatul** generat de comportament
- **Proces**
 - Descompunerea/Detalierea unei **functionalitati**
 - ORDONAREA TEMPORALA SAU LOGICA DE EVENIMENTE CARE REALIZEAZA O FUNCTIE SAU MAI MULTE
 - **Automat** => **nu exista nicio implicare umana in initiere sau executie**
 - Poate permite interventia umana pentru a trece peste decizii
 - **Semi-automat**: o parte e automatizat
 - **Manual**: nu exista ajutor semnificativ de la calculator
- **Resursa**
 - Orice persoana, dispozitiv, sistem software, materiale care efectueaza functii
 - **Activa = actor**
 - **Agent** = resursa activa de regula sistem software
 - **Resursa pasiva** = materiale, echipamente sau info folosite de resurse active
- **Actiune comuna**
 - Orice mod productiv in care mai multe resurse participa activ la realizarea unei functii
- **Informatia**
 - Secventa de simboluri interpretata ca mesaj
 - Eveniment care afecteaza starea unui sistem dinamic
- **Comunicatia**
 - Orice operatie care are ca rezultat un flux de informatii de la un actor la altul
 - Schimbul de date între actori în exercitarea unei acțiuni comune
- **Mecanism de comunicare**
 - Modul in care are loc transferul de info
- **Protocol de comunicare**
 - Regulile detaliate ale fluxului de info
- **Resurse de comunicare**
 - Fisiere, baza de date, mesaje, semnale

Auto-integrarea:

- Cerintele tehnice de comunicare sunt solutionate in mod automat de catre agentul de comunicare

Sistem auto-integrator:

- Un sistem capabil de a se reconfigura si/sau a configura comunicatiile astfel incat sa coopereze cu alte sisteme
- Sistem prevazut cu “**mecanisme de invatare**”

Sistem auto-descriptiv:

- Cerinte tehnice
 - Tehnologie comuna pentru transmiterea si primirea datelor
 - Protocoale comune
 - Structura si reprezentare comuna pentru date
- = un sistem auto-integrator conectat la alte sisteme care isi **publica capabilitatile** si determina in mod automat **rolurile** celorlalte sisteme
- Cu scop de a facilita re folosirea componentelor

Sisteme informatice integrate:

- **Proiectarea:**
 - Intelegerea arhitecturii de ansamblu a proceselor si datelor existente
 - Identificarea componentelor care necesita schimb de info
 - Utilizarea tehnologiilor moderne

Ingineria proiectării sistemelor compozite:

- Rezultat -> specificatia sistemului ce satisface cerintele formulate
- = secventa recursiva:
 - DECOMPOZITIE
 - Cerinta functionala/ de comportament => set de sub-comportamente
 - Fiecare sub-comportament e atribuit unui “dispozitiv”
 - Idee de a avea sub-comportamente care pot fi realizate printr-un dispozitiv/clasa cunoscuta
 - SINTEZA
 - Se realizeaza proiectul setului de dispozitive
 - COMPOZITIA
 - Se inglobeaza sub-componentele intr-un singur dispozitiv
 - ANALIZA
 - Se testeaza modul in care proiectul raspunde cerintelor
 - Conflicte
 - Compromisuri
 - Ordinea depinde de expertiza inginerului
- PRINCIPII: **uniformitate, modularizare, structurare, implementare, documentatie, imagine de ansamblu**

Aplicatii compozite:

- CA = notiune integratoare pentru principiile moderne de dezvoltare de soft
- 3 niveluri de abstractizare tehnica:
 - Modelarii logice a sistemului
 - Modelarii functionale
 - Modelarii tehnice a sistemului
- **Gui**
 - Nivel prezentare
 - Compozitie - arhitectura de sistem
 - Interactiunile cu userul
- **Procese**
 - Nivel aplicatie
 - Compozitie
 - Rol important la nivelul modelarii - descrie functionalitatile viitorului sistem
- **Workflow**
 - Nivel aplicatie
 - Compozitie, invocare
 - Ajutor in implementarea proceselor de afaceri specifice
 - Detaliaza modelul tranzatiilor
- **Componente**
 - Nivel domeniu/infrastructura
 - Invocare
 - In aplicatii tip SOA, serviciile sunt componente
- **Reguli**
 - Domeniu
 - Invocare
 - Parte din logica activitatilor si a proceselor
- **Date**
 - Domeniu
 - Invocare
 - Modelul datelor
- **Grupuri si roluri**
 - Infrastructura
 - Infrastructura
 - Managementul identitatii si accesului la sistem
- **TIPURI DE ARHITECTURI:**
 - Centralizata
 - two-tier : logic + GUI -> data
 - Three-tier: GUI -> logic -> data
 - N-tier
 - Service Oriented Architecture (SOA)
 - Arhitectura client server orientata pe servicii
 - Arhitectura sistemelor – Microservicii
 - Pot fi dezvoltate si publicate independent
 - Prin api gateway

Probleme de integrare:

- **Interoperabilitatea** = capacitatea de a furniza/ accepta servicii altor alte sisteme/componente
- **Modularitate** = capacitatea de a delimita capabilitatile sistemului in blocuri functionale
- **Integrabilitate** = capacitate de a incorpora componente
- Cand avem mai multe componente/ niveluri arhitecturale
- **DE NATURA TEHNICA:**
 - Nepotriviri intre **comportamnetul** si **reprezentarea** informatiilor la nivelul unei interfete
 - Probleme de interoperabilitate
 - Solutie:
 - IMPACHETARE=> WRAPPERS
 - MODIFICARE COMPONENTEI
 - **Conflicte de conexiune**
 - 2 resurse nu au aceleasi specificatii tehnice pentru a se conecta
 - **La nivel de mecanism de comunicare**
 - Nu folosesc acelasi mecanism de comunicare
 - **La nivel de protocol**
 - Nu se potrivesc protocoalele a nivel aplicatie
 - **SOLUTIE:**
 - Modificare mecanismelor sau protocoalelor
 - **Conflicte sintactice**
 - Structuri de date si reprezentari diferite
 - **SOLUTIE:**
 - Componenta intermediara care "traduce"
 - Translator
 - Un limbaj
 - Construirea componentei intermediare
 - **Conflicte de control**
 - Componentele se bazeaza pe ipoteze diferite legate de fluxul controlului
 - 2 componente nu cad de acord cu ROLUL lor
 - Probleme:
 - **Prea multi lideri**
 - Ex: nicio componenta nu si asigura rolul de server si nu raspunde la cererilor clientilor
 - **SOLUTIE**
 - Un client ia rolul de server (componenta intermediara)
 - **Niciun lider**
 - Ambele componente iau rolul de server
 - **SOLUTIE**
 - Adauga o componenta care implemneteaza schimbul: client pull pentru serverul pull, client push pentru serverul push

- **Conflicte legate de calitatea serviciului**
 - O componenta nu satisface cerintele tehnice care se refera la performanta, siguranta, etc.
 - Ex: clientul cere un rezultat cu 10 cifre cifre dupa virgula, dar serverul da re numere cu 7 cifre dupa virgula
 - **SOLUTII**
 - IN GENERAL MODIFFICARE COMPONENTELOR
 - Alta solutie: **wrapper, relaxarea cerintelor**
- **Conflicte legate de consistenta datelor**
 - Datele intre 2 componente sunt accesate la momente de timp diferite si nu sunt consistente => race condition
 - **SOLUTIE**
 - Fortarea unei componente sa reciteasca datele
 - Sincronizare
 - Adaugare structura informationala in comunicarea intre ele
- **Conflicte semantice**
 - Neconcordanta in comunicarea dintre componente
 - Tipuri:
 - **Conceptualizare**
 - Componentele au interfata implementata incompatibil
 - **SOLUTIA:**
 - Folosirea aceleiasi implementari
 - **Domeniu de aplicare**
 - Un concept nu e prezent in modelul interfetei celeilalte componente
 - Ex: mouse cu 3 butoane comunica cu un mouse cu 2 butoane
 - **SOLUTIA**
 - Info inutile pot fi ignorate
 - Trebuie restul functionalitatilor sa fie furnizate de catre o alta componenta add-on
 - **De interpretare**
 - **SOLUTIE:** wrapper, interfete standard
 - **De referinta**
 - Diferite sisteme de referinta
 - Ex: cerc vs elipsa
 - Strans legate de conflicte sintactice
 - **SOLUTII:** Wrapper
- **Conflicte functionale**
 - Comportamentul unei resurse difera de cel asteptat de catre alta resursa
- **LEGATE DE POLITICA SISTEMULUI**
 - Necorelare suficienta intre comportamentul sistemului si functionalitatile asteptate: securitate, corectitudine, performanta
 - Compromisuri

- **LEGATE DE LOGISTICA**
 - Strans legat de partea de management

Procesul de integrare:

- **Intelegerea domeniului organizatiei**
 - Analiza **proceselor de afaceri** => problema de identificare a **cerintelor**
 - Interfatare cu **documente, persoane, sisteme**
- **Intelegerea datelor**
 - Definirea modului de transfer
 - 3 pasi:
 - **Identificarea datelor**
 - Etapa 1: crearea unei liste a sistemelor sursa de date
 - Etapa 2: identificare, proprietarul, localizarea fizica, info de proiectare
 - **Formatul datelor**: lungime, tip, nume
 - **Latenta** : cat de des trebuie actualizate: timp real, aproape timp real, unice
 - **Aspecte de integritate**
 - Intelegerea regulilor la construirea bazei de date
 - Principiile ALCOA+
 - **Catalogarea datelor**
 - Colectare de metadate si alte date din domeniu
 - Info despre sisteme, securitate, procese conectate
 - **Construirea modelului metadatelor**
 - Descrie structura datelor existente in organizatie si modul in care acestea interactioneaza
 - Un repository de metadate contribuie la integrarea la **nivelul de date** si constituie baza pentru alte niveluri de integrare
 - **Modelul fizic**: modelul logic, catalogul datelor
 - Normalizarea organizatiei
 - Descompunerea structurilor de date complexe in relatii simple
 - = reducerea volumului de date redundante
 - La nivel logic, la nivel fizic, in cazul proiectelor de integritate
- **Intelegerea proceselor de afaceri**
 - DE AFACERE (Business processes)
 - Descrie prin **listarea functiilor** sau metodelor oferite de o aplicatie
 - Sunt bine documentate
 - MODELARE (BPMN)
 - Limbaj grafic de definire a proceselor
 - INTEGRAREA
 - Se dezvoltă o perspectiva la nivelul **metodelor** (=> **intocmirea unui catalog al proceselor**) si **proceselor**

- Se documenteaza toate procesele si modul cum relationeaza
- **Modelul de afaceri** = rezultatul agregarii = **studiul final al integrarii la nivel de metoda** => axat pe intelegerea proceselor
- Scopul final e de a depasi simpla conexiune intre aplicatii si de a ajunge la definirea si proiectarea unui flux al proceselor de afaceri
- Paradigma independenta de implementare (nu prea inteleg slide urile astea)
- Reprezentare proceselor in mod mai structural => **MODELE**
 - = **gasirea unor structuri comune**
 - **Arhitecturale** - structura de baza a sistemului (architectural patterns)
 - **De proiectare** - definirea componentelor si a interactiunilor (design patterns)
- **Identificarea interfetelor aplicatiilor:**
 - Pentru a le integra la nivelul interfetelor aplicatiilor (API)
 - **NECESAR:** crearea unui director al interfetelor aplicatiilor
 - Gandit ca o lista de procese de afaceri disponibile intr o aplicatie
 - **ESENTIAL:** intelegerea proceselor de afaceri
 - **SEMNTICA APLICATIEI** = ce inseamna un anumit lucru intr-o aplicatie
- **Etape aditionale:**
 - Identificarea evenimentelor din sistem
 - Identificarea scenariilor de transformare a datelor
 - Maparea transferurilor de informatie
 - Utilizarea tehnologiilor
 - Testarea
 - Performanța
 - Evaluarea rezultatelor
 - Definirea procedurilor de întreținere

Puncte de vedere:

- **Intreprindere:** comportamentele relative la contextul **organizatiei**
- **Informational:** **informatiile**, intrebuintarea si inretpretarea lor
- **Computational:** **descompunerea** sistemului in componente specifice care interactioneaza intre ele prin interfete
- **Ingineresc:** dezvolta **mecansme** menite sa suporte interactiunea intre componentele computationale
- **Tehnologic:** alege cat mai precis tipul de **tehnologie** pentru implementarea sistemului si a comunicarii intre componentele lui

Aspecte de integrare:

- = un set de teme ce trebuie avute in vedere la integrare privind compatibilitatea
- Tipuri:
 - **Tehnice:**
 - Perspective tehnlogice si ingineresti

- **De integrare a coordonarii** = intelegere intre componente
 - **De integrare a datelor** = componentele se pun de acord asupra organizarii
- **De politica**
 - Capacitatea sistemului de a derula procesele de afaceri
 - Conform politicilor companiei
- **Logice**
 - Impactul pe care il are sistemul asupra afacerii in sine
- **Semantice**
- **Functionale**
- **MODELUL DE AFACERI:**
 - Include modelul **proceselor** si modelul **datelor**
 - = corespondenta dintre nivelurile de abstractizare
 - Se construiesc in etapa de analiza a afacerii
- **MODELAREA DOMENIULUI:**
 - **domeniul**= reuneste continuturi, concepte si idei necesare prentu **descrierea** arhitecturii unui sistem
 - Criterii de descriere: perspectivele si modelul folosite
 - = descriere cuprinzatoare
 - **CARACTERISTICI**
 - Reutilizabilitate
 - Portabilitate
 - Interopetabilitate = descrieri implicite si formale ale interfetelor
 - **INGINERIA** = O procedura de stabilire a cerintelor domeniului: analiza, proiectarea si implementarea domeniului
 - Programarea generativa= proceduri prin care se realizeaza componente care pot rula automat / se realizeaza cod compilat din domeniul modelului = generatoare
 - **DDD** = paradigma care ajuta la creerea sistemului pornind de la cerintele de business

Modelare semantică:

- = ne ajuta sa definim **date** si **relatii** intre entitati
- Abilitatea de a **abstractiza** diferite tipuri de date si entitati
- **Ontologie**
 - Vocabular al modelului semnat pe baza caruia putem face interogari definite de utilizator
 - Rezultatul e un catalog de tipuri de lucruri care se presupune ca exista in domeniul de interes D din perspectiva unei persoane care utilizeaza limbajul L
 - **IMPORTANTA:**
 - Clarifica structura cunostintelor => metamodel
 - Partajeaza cunostintele
 - Analiza unui domeniu de cunoastere
 - **E UN GRAFIC ORIENTAT ACICLIC CU UN SINGUR NOD RADACINA**
 - Nod = concept -> o unitate la care cineva s -ar gandii

- Arce
 - Cele mai importante IS-A
- **Semantic web**
 - Permite descrierea formală a resurselor existente pe Internet
 - MOTIVATIE
 - Identificarea rapidă și precisă a resurselor relevante pentru user
 - Layer cake
 - URI
 - RDF
 - Resursa: subiect identificabil prin URI
 - Proprietate: predicat ce reprezintă relația dintre resurse
 - Valoare: informația asociată cu o anumită resursă
 - OWL, SPARQL

Domenii de abstractizare:

- **Informational**
 - Modele **semantice, informationale, de date**
 - Descrie **obiectele** din **procesele de afaceri**, relațiile dintre ele și proprietățile acestora
 - Model **conceptual** => nu ține cont de implementare
 - Diagrama entitate-relație
 - Definiște structura logică a bazei de date => **corespondența directă cu schema bazei de date**
 - **Modelul semantic**
 - = model conceptual
 - Descrie prin reprezentare grafică înțelesul obiectelor
 - Descrie semnificația termenilor
 - Pentru AI
 - **Modelul datelor**
 - = model tehnic
 - Definiște organizarea și reprezentarea info
 - Include modelul de baze de date, limbaje de transfer, struct de date
 - Diagrama bază de date
- **Functional**
 - Descriu comportamentul sistemului și al componentelor
 - Comportamentul = ceea ce face sistemul
 - Funcția = rezultatul comportamentului
 - Cuprinde modele comportamentale din uml
 - **Model funcțional** -> Reprezentare a **funcției** = diagrama uml **use case** => **mod incomplet**
 - **Model de execuție** -> modificările sistemului datorită unor acțiuni = diagrame uml de secvență, de interacțiune, de activități
 - **Model de stare** -> comportamentul sistemului ca răspuns la activități = diagrama de stare

- **Modelul de proces** -> descompunerea unei functii (cum modelul in sistemul indeplineste o functie)
- **De coordonare**
 - **Model de interfatare**
 - Mecanismul prin care o componenta contribuie la interactiune
 - **Model de coordonare**
 - Fluxul de info si control
 - **Protocoalele**
 - Reguli de implementare si functionare al nivel de interfata
 - **Model de prezentare**
 - Cazuri particulare un componentele implica actiune umana
- **De sistem**
 - **Model de structura**
 - Organizarea resurselor in sistem, inclusiv rolurile componentelor
 - **Modele de politica**
 - = Model conceptual
 - Specifica regulile pe baza carora se deruleaza procesele de afaceri
 - In special cele pentru siguranta sistemului, a securitatoo
 - **Modele de retea**
 - Specifica structura logica si fizica a sistemului din perspectiva comunitatii

Ingineria sistemelor:

- = orice abordare metodica de sinteza a unui sistem care
 - Defineste perspectivele sistemului
 - Gestioneaza relatia intre cerinte
- Orientata spre specificarea problemei
- Ofera un plan de organizare a info si de directioanre a activitatilor = **procesul de inginerie a sistemelor**
 - Identifica punctele de vedere relevante (**instrument de integrare**)
 - Analizeaza cerintele (**instrument de orchestare**)
 - Atribuire task-uri
 - Evalueaza masurile (**instrument de validare**)
 - **Conceptul** sistemului = abstractizare
 - **Solutia** sistemului = o instanta reprezentativa a conceputului sistemului
- SPECIFICAREA SI MODELAREA
 - **Modelarea specificatiilor**
 - Cerinte relevante din **perspectiva operatioanala**: performanta, securitate...
 - Cerințe relevante din **perspectiva dezvoltării**: scalabilitate ...
 - Gestionarea produsului software
 - Product management
 - Caracteristici calitative: capacitate, maturitate
 - Cerinte tehnice /non-functionale: perspectiva operatioanala, de dezvoltare

- **Modelarea functionala (R)**
 - Un model functional = reprezentare structurala a functiilor
 - Modele comportamentale din uml
 - Vizeaza
 - Functionalitate
 - Date de intrare
 - Date de iesire
 - Granularitate
 - Functionalitati primare, secundare, elementare
 - Modelarea **activitatilor**
 - Succesiunea pasilor printr-un grafic partial ordonat
 - Fiecare task e impartit in subtask-uri
 - Administrativa sau tehnica
 - Ex: WBS, PERT, CPM
 - **De executie:** modificarea sistemului dupa niste actiuni
 - **De stare:** comportamentul sistemului ca raspuns la activitati
 - Modelarea **proceselor**
 - Forma speciala de modelare a activitatilor
 - Se axeaza pe interactiuni
 - = descompunerea unei functii, modul in care un sistem indeplineste o functie
 - Tehnica sau administrativa
 - BPMN, IDEF0
 - Retele petri
 - Metode formala
 - Pt modelarea si verificarea sistemelor dinamice cu evenimente discrete
 - **Graf bipartit**
- **CERINTE:**
 - **funcționale** – descriu funcționalitatea sistemului (use case)
 - **juridic-contractuale** – clauze de întreținere, drept de proprietate intelectuală, versiuni ulterioare – update-uri
 - **tehnologice** – scalabilitate, extensibilitate, etc
 - **de calitate** – durata maximă de nefuncționare, etc.
 - **asistarea utilizatorului** – legătura dintre pagini, workflow
 - **modularizare** – ce pachete se livrează
 - **activități** – planificare